



COMPUTER ENGINEERING PROGRAM
FACULTY OF ENGINEERING, THAI-NICHI INSTITUTE OF TECHNOLOGY

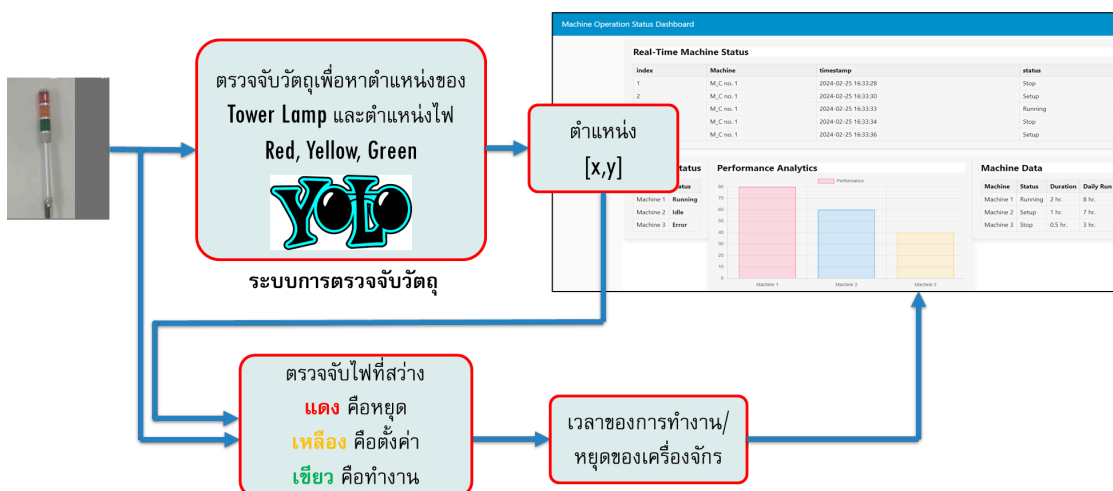
ปีการศึกษา : 2/2567
หลักสูตร : Special Topic in AI and IoT on Raspberry Pi
ผู้สอน : อาจารย์อัฒนา เชนโตะ
การทดลองที่ 09 : การตรวจจับวัตถุในพื้นที่ที่กำหนดด้วย SmartBoxAi Part I

1. วัตถุประสงค์

- 1.1 เพื่อทำความเข้าใจเกี่ยวกับการรวบรวมชุดข้อมูลสำหรับฝึกสอนเบื้องต้น
- 1.2 เขียนโค้ด python บนบอร์ด Raspberry Pi เพื่อสร้างระบบการตรวจจับวัตถุเข้าพื้นที่ต้องห้ามด้วย SmartBoxAi

2. รายละเอียด

การพัฒนาระบบตรวจจับวัตถุด้วย YOLO (You Only Look Once) ร่วมกับ Node-RED เป็นกระบวนการที่ช่วยเพิ่มประสิทธิภาพในการประมวลผลและจัดการข้อมูล IoT ได้อย่างชาญฉลาด ขั้นตอนแรกเริ่มจากการเตรียมสภาพแวดล้อม โดยติดตั้ง Python และ YOLO (เช่น YOLOv5 หรือ YOLOv8) พร้อมกับ Node-RED สำหรับจัดการการไหลของข้อมูล ถัดมาเป็นการรวบรวมและติดป้ายกำกับข้อมูล (Annotation) เพื่อฝึกโมเดล YOLO ให้ตรวจจับวัตถุได้แม่นยำ หลังจากฝึกโมเดลเสร็จสิ้น จะได้ไฟล์โมเดล (เช่น best.pt) ซึ่งสามารถนำไปใช้งานร่วมกับ Node-RED ได้โดยใช้ Node-RED Dashboard เพื่อสร้างอินเทอร์เฟซในการแสดงผล ในส่วนของ Node-RED จะมีการสร้าง flow สำหรับรับภาพจากกล้องหรือไฟล์ภาพแล้วส่งไปยัง YOLO ผ่าน Node.js หรือ Python API เพื่อประมวลผลและส่งผลลัพธ์กลับมาแสดงบนแดชบอร์ด โดยสามารถแสดงตำแหน่งวัตถุในภาพ พร้อมกับข้อมูลจำแนกประเภทแบบเรียลไทม์ นอกจากนี้ยังสามารถเชื่อมโยงกับอุปกรณ์ IoT อื่น ๆ เช่น เซ็นเซอร์ หรือระบบแจ้งเตือน เพื่อสร้างระบบอัตโนมัติที่ตอบสนองตามข้อมูลที่ตรวจจับได้ ซึ่งกระบวนการนี้ไม่เพียงแต่ช่วยในการพัฒนาระบบตรวจจับวัตถุ แต่ยังขยายไปสู่การจัดการข้อมูลและควบคุมอุปกรณ์ IoT ได้อย่างยืดหยุ่นและทรงพลัง



เตรียมซอฟต์แวร์สำหรับภาคปฏิบัติ

แพลตฟอร์มที่ต้องการใช้งานมีดังต่อไปนี้

- ☐ ใช้ระบบ Raspberry Pi OS
- ☐ ใช้ Thonny/Terminal บน Raspberry Pi OS เพื่อเขียนโค้ด
- ☐ ใช้ภาษาโปรแกรม **Python version.....3.9.2....**
- ☐ ใช้โมดูล **OpenCV version...4.10.0.....**เพื่อช่วยการใช้งานด้านภาพหรือวิดีโอ
- ☐ ใช้โมดูล **Pytorch version...2.4.1....**เพื่อสร้างโมเดลปัญญาประดิษฐ์
- ☒ ใช้โมเดล **YOLO version...8.2.95.....**สำหรับการตรวจจับวัตถุ
- ☐ ใช้โมเดล **NODE_RED version...4.0.8.....**สำหรับการสร้างระบบตรวจจับวัตถุ

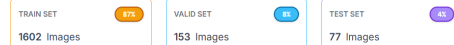
หมายเหตุ การตรวจสอบ version ของ node-red ด้วย cmd ดังนี้

```
npm -g info node-red version
```

แบบฝึกหัดที่ 1: ดำเนินค้นหาโมเดลฝึกสอนตรวจจับวัตถุที่ดีที่สุดด้วย YOLOv8

- 1) หลังจากที่ได้เตรียมข้อมูลบน **RoboFlow** หรือ **Makesense** เป็นที่เรียบร้อยแล้วโดยให้สรุปดังตารางข้างล่างนี้ ก่อนการดำเนินงานส่วนถัดไป

ตารางชุดข้อมูลเวอร์ชันที่ 1

ลำดับ	รายการ	จำนวน (รูป)	สัดส่วน Train-Validation-Test
1	จำนวนข้อมูลทั้งหมด	1831	
2	จำนวนข้อมูลคลาสที่ 1 Guide Signs	363	-
3	จำนวนข้อมูลคลาสที่ 2 Regulatory Signs	1199	-
4	จำนวนข้อมูลคลาสที่ 3 Warning Signs	956	-
5	แหล่งที่เก็บชุดข้อมูล	link: https://drive.google.com/drive/folders/1f5RDhA1QlHpJl3O3laU0ObqfWS3OfSAk?usp=drive_link	

Progress

34 Images

34 Annotated

0 Unannotated

Instructions

Edit

No specific instructions were added when this job was assigned

Assignment

Reassign

Adna Sento

Labeler

Timeline

?

\$Job created via API and assigned it to adna@tnl.ac.th.

1/27/2022 10:29:17 AM

Unannotated 0

Annotated 34

Sort By Newest

ชุดข้อมูลของsign

SIGN2GETHER

sign2gether

Object Detection

DATA

Upload Data

Annotate

Dataset 1831

Folder: datasets

Progress

1831 Images

Instructions

Edit

No specific instructions were added when this job was assigned

Assignment

Kachachan Ruanpetch

Labeler

To Do 0

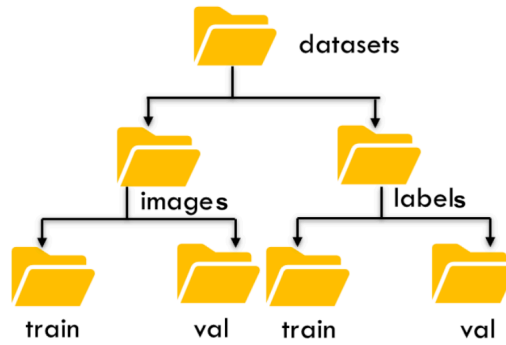
Approved 1830

Annotated 1

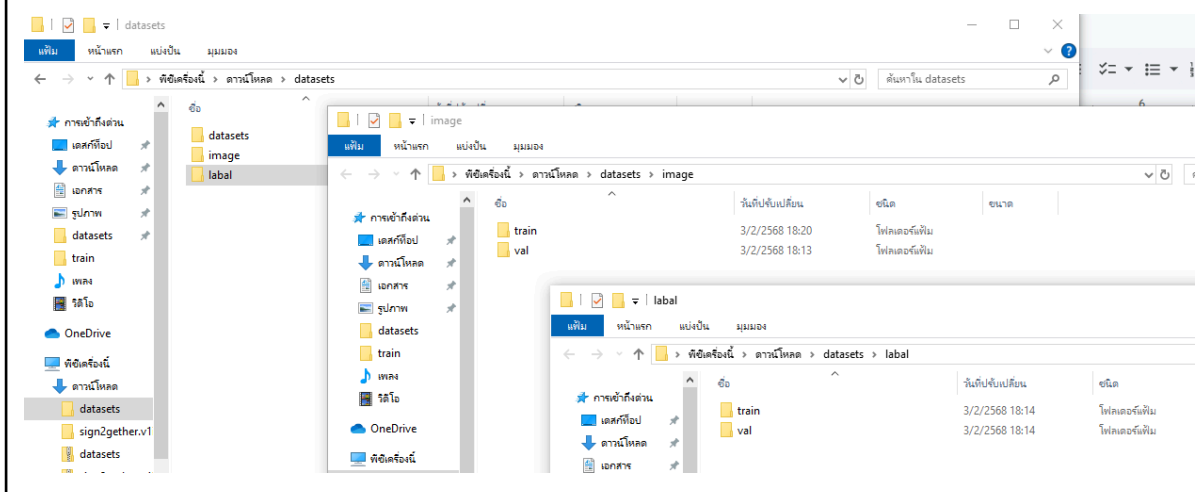
1689577752065_png

2) จากเว็บ [RoboFlow](#) หรือ [Makesense](#) ให้ดาวน์โหลด Datasets ด้วย Format ของ YOLOv8 ซึ่งเมื่อดาวน์โหลดแล้วให้ตรวจสอบโฟลเดอร์สำหรับฝึกสอน ซึ่งประกอบด้วย 2 โฟลเดอร์ย่อยได้แก่ โฟลเดอร์ images และ โฟลเดอร์ labels ซึ่งแต่ละโฟลเดอร์จะประกอบด้วยดังนี้

- โฟลเดอร์ images สำหรับเก็บไฟล์ภาพ ประกอบด้วย 2 โฟลเดอร์ย่อยดังนี้
 - train สำหรับเก็บไฟล์ภาพของการชุดข้อมูลสำหรับฝึกสอน
 - val สำหรับเก็บไฟล์ภาพของการชุดข้อมูลสำหรับการปรับปรุง
- โฟลเดอร์ labels สำหรับเก็บไฟล์ annotation ประกอบด้วย 2 โฟลเดอร์ย่อยดังนี้
 - train สำหรับเก็บไฟล์ภาพของการชุดข้อมูลสำหรับฝึกสอน
 - val สำหรับเก็บไฟล์ภาพของการชุดข้อมูลสำหรับการปรับปรุง



โฟลเดอร์สำหรับฝึกสอนโปรเจค



- 3) ให้เชื่อมต่อ Google Drive และเมื่อดำเนินการเตรียมข้อมูลเป็นที่เรียบร้อยแล้ว ให้ตั้งค่า ส่วนของเฟรมเวิร์ค YoLov8 บน Google Colab เพื่อติดตั้ง YoLov8 ดังนี้

```

!pip install ultralytics==8.2.103 -q

from IPython import display
display.clear_output()

import ultralytics
ultralytics.checks()
from ultralytics import YOLO

from IPython.display import display, Image
  
```

ผลลัพธ์ของการฝึกสอน

```

/content/datasets
_____ 80.3/80.3 kB 5.4
_____ 66.8/66.8 kB 4.1
_____ 54.5/54.5 kB 3.1
visit https://app.roboflow.com/auth-cli to get your authentic:
Paste the authentication token here: .....
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in football-players-obj-det
  
```

```

!pip install ultralytics==8.2.103 -q

from IPython import display
display.clear_output()

import ultralytics
ultralytics.checks()
from ultralytics import YOLO

from IPython.display import display, Image

Ultralytics YOLOv8.2.103 Python-3.10.12 torch-2.5.1+cu121 CUDA:0 (Tesla T4, 15995MiB)
Setup complete (2 CPUs, 12.7 GB RAM, 32.7/112.6 GB disk)
  
```

- 4) ป้อนคำสั่งเพื่อฝึกสอน ดังนี้

```
!yolo task=detect mode=train model=yolov8s.pt
data='/content/drive/MyDrive/towerlamp/datasets/data.yaml'
epochs=10 imgsz=640 plots=True
```

ผลลัพธ์ของการฝึกสอน

10 epochs completed in 0.026 hours.
Optimizer stripped from runs/detect/train/weights/last.pt, 22.5MB
Optimizer stripped from runs/detect/train/weights/best.pt, 22.5MB

Validating runs/detect/train/weights/best.pt...

Ultralytics YOLOv8.2.103 Python-3.11.11 torch-2.5.1+cu124 CUDA:0 (Tesla T4, 15095MiB)

Model summary (fused): 168 layers, 11,126,745 parameters, 0 gradients, 28.4 GFLOPs

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	100	460	0.471	0.572	0.481	0.335
Football-footballplayers	23	23	0.663	0.478	0.572	0.331
player	92	421	0.593	0.924	0.788	0.686
referee	14	16	0.157	0.312	0.0818	0.069

Speed: 0.2ms preprocess, 3.9ms inference, 0.0ms loss, 3.5ms postprocess per image

Results saved to runs/detect/train

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
10/10	4.18G	1.063	1.595	1.314	22	640: 100% 5/5 [00:01:00:00, 3.46it/s]
Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	1602	2133	0.314	0.33	0.238	0.118

10 epochs completed in 0.076 hours.

Optimizer stripped from runs/detect/train/weights/last.pt, 22.5MB

Optimizer stripped from runs/detect/train/weights/best.pt, 22.5MB

Validating runs/detect/train/weights/best.pt...

Ultralytics YOLOv8.2.103 Python-3.11.11 torch-2.5.1+cu124 CUDA:0 (Tesla T4, 15095MiB)

Model summary (fused): 168 layers, 11,126,745 parameters, 0 gradients, 28.4 GFLOPs

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	1602	2133	0.275	0.351	0.252	0.126
Guide Signs	283	310	0.394	0.197	0.240	0.137
Regulatory Signs	774	908	0.202	0.586	0.321	0.159
Warning Signs	668	825	0.23	0.269	0.187	0.0819

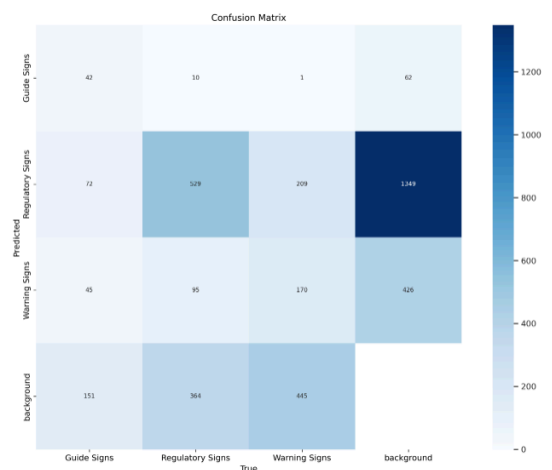
Speed: 0.3ms preprocess, 4.5ms inference, 0.0ms loss, 3.4ms postprocess per image

Results saved to runs/detect/train

5) ป้อนคำสั่งเพื่อประเมินผลของการฝึกสอนด้วยตาราง Confusion Matrices ดังนี้

```
Image(filename=f'/content/runs/detect/train/confusion_matrix.png', width=600)
```

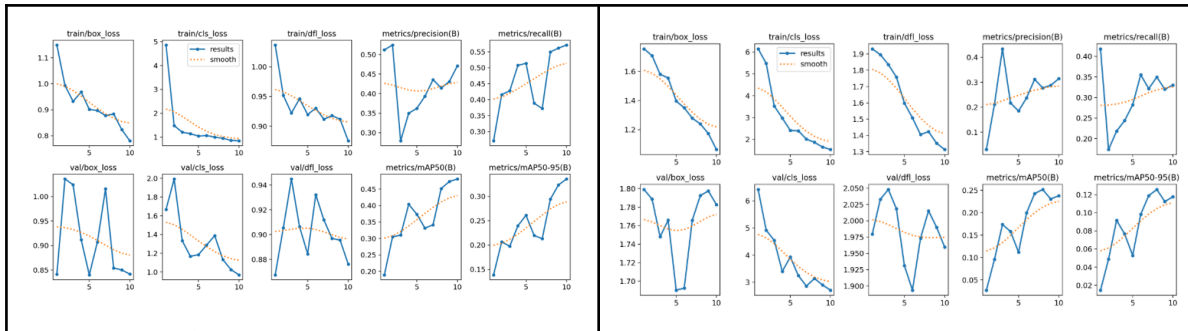
ผลลัพธ์ของการฝึกสอน



6) ป้อนคำสั่งเพื่อประเมินผลของการฝึกสอนด้วยกราฟ accuracy, losses ดังนี้

```
Image(filename=f'/content/runs/detect/train/results.png', width=600)
```

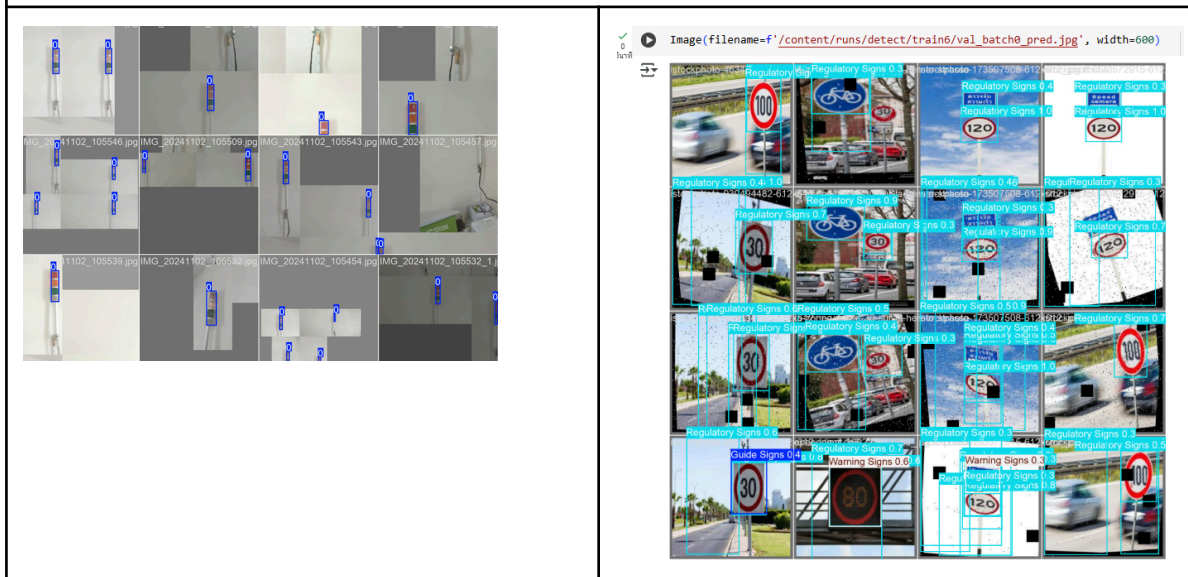
ผลลัพธ์ของการฝึกสอน



7) ป้อนคำสั่งเพื่อประเมินผลของการฝึกสอนด้วยภาพ ดังนี้

```
Image(filename=f'/content/runs/detect/train/val_batch0_pred.jpg', width=600)
```

ผลลัพธ์ของการฝึกสอน



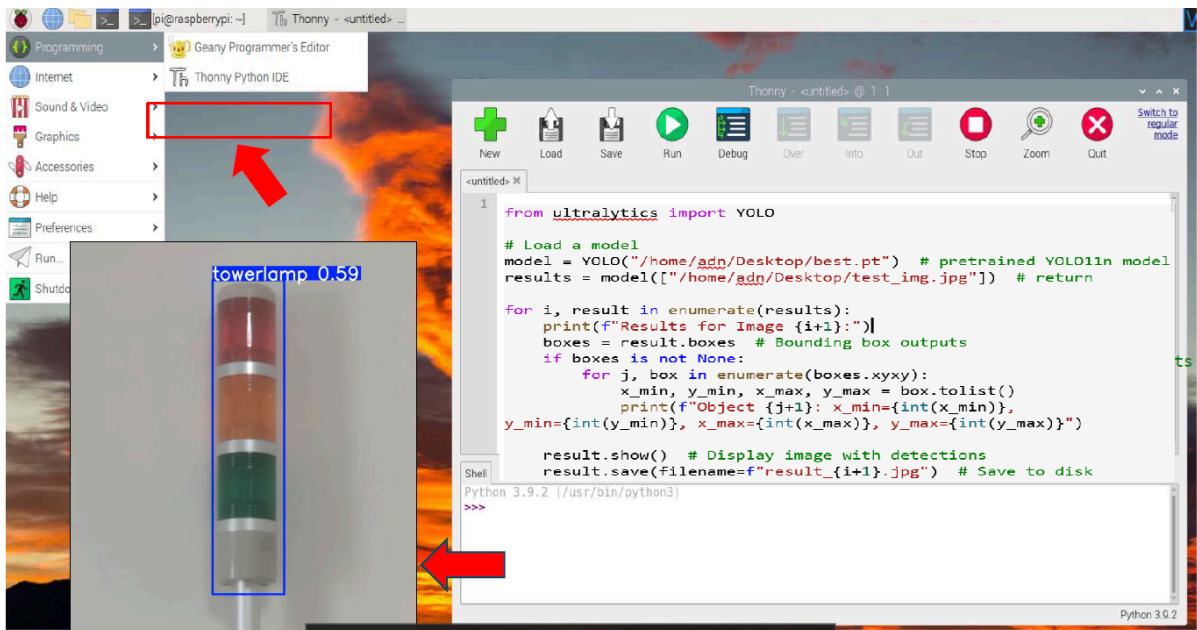
8) ดาวน์โหลดไฟล์ 'best.pt' ซึ่งเป็นไฟล์ของ trained model ที่ได้จากการฝึกสอนก่อนหน้านี้ (สามารถไปยังโฟลเดอร์ '`/content/runs/detect/train/weight`' บน Google Colab และป้อนโค้ด Python นี้เพื่อใช้งานบนบอร์ด Rpi ดังนี้

```
from ultralytics import YOLO

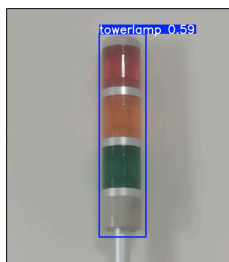
# Load a model
model = YOLO("/home/adn/Desktop/best.pt") # pretrained YOLO11n model
results = model(["/home/adn/Desktop/test_img.jpg"]) # return

for i, result in enumerate(results):
    print(f"Results for Image {i+1}:")
    boxes = result.boxes # Bounding box outputs
    if boxes is not None:
        for j, box in enumerate(boxes.xyxy):
            x_min, y_min, x_max, y_max = box.tolist()
            print(f"Object {j+1}: x_min={int(x_min)}, y_min={int(y_min)}, x_max={int(x_max)}, y_max={int(y_max)}")
```

```
result.show() # Display image with detections
result.save(filename=f"result_{i+1}.jpg") # Save to disk
```



ผลลัพธ์ของการฝึกสอน



แบบฝึกหัดที่ 2: งาน Develop โมเดลการฝึกสอนสำหรับใช้งานบนบอร์ด Raspberry Pi

ตารางบันทึกผลการทดลองเพื่อหาโมเดล AI ที่ดีที่สุด (google collab)

Exp. No.	Model	Epoch	Batch Size	Precision	Recall	mAP50	mAP50-95
1	Yolov8s	100	16	0.613	0.464	0.461	0.218
2	Yolov8s	200	16	0.631	0.468	0.484	0.233
3	Yolov8s	300	16	0.649	0.472	0.507	0.248
4	Yolov8s	400	16	0.667	0.476	0.530	0.263
5	Yolov8s	500	16	0.685	0.480	0.553	0.278
6	Yolov8s	600	16	0.703	0.484	0.576	0.293

ก จำนวนทั้งหมด Model ที่ทำการฝึกสอน

พารามิเตอร์พื้นฐานสำหรับการฝึกสอน

พารามิเตอร์	คำนิยาม	ค่าเริ่มต้น
epochs	จำนวนรอบการฝึกสอนโมเดล ยิ่งมาก โมเดลจะเรียนรู้มากขึ้น แต่ก็ใช้เวลาเพิ่มขึ้น	100
batch	ขนาดของชุดข้อมูลที่ใช้ในการฝึกต่อรอบ ยิ่งมากช่วยให้โมเดลเรียนเร็วขึ้น แต่ใช้ RAM มากขึ้น	16
imgsz	ขนาดของภาพอินพุต (เช่น 640x640) ขนาดที่ใหญ่ขึ้นอาจช่วยให้ตรวจจับแม่นยำขึ้นแต่ช้าลง	640
lr0	ค่าเริ่มต้นของอัตราการเรียนรู้ (Learning Rate)	0.01
optimizer	ตัวเลือกอัลกอริธึมสำหรับการฝึกสอน (SGD, Adam, AdamW)	SGD
Model	โมเดล เช่น Yolov8n, Yolov8s, Yolov8m, Yolov8l, Yolov8x	Yolov8n

ตัวอย่างที่ 3: ฝึกโมเดลโดยใช้ Optimizer AdamW และลด Learning Rate

```
yolo train model=yolov8m.pt data=data.yaml epochs=200 batch=32 imgsz=640  
optimizer=AdamW lr0=0.005
```